

Tập 10 - Calico - Architecture

Kiến trúc Calico: Felix, BIRD, Typha — Ai làm gì?

Phần 2 — Calico · `#calico` `#felix` `#BIRD` `#typha` `#architecture`



CALICO

BY TIGERA

Mục tiêu tập này

- Hiểu vai trò chính xác của **Felix**, **BIRD**, **Typha** trong kiến trúc Calico.
- Trace luồng từ `NetworkPolicy` YAML → iptables rule trên Node.
- Quan sát Felix log khi policy thay đổi — event-driven trong ms.
- Dùng `calicoctl` để debug workload endpoints, IP pools, BGP status.

Prerequisites: Cluster từ Tập 9 với Calico đang chạy.

Luồng dữ liệu trong Calico

NetworkPolicy (YAML)



K8s API Server
watch



Typha (nếu cluster lớn) ← cache, fan-out tới nhiều Felix



Felix (mỗi Node) ← dịch policy → rules



├── iptables chains (cali-FORWARD, cali-fw-*, cali-tw-*)
└── eBPF maps (nếu dùng eBPF dataplane)

BIRD (mỗi Node) ← quảng bá Pod subnet qua BGP



└── Kernel routing table

Felix: Policy Engine trên từng Node

Felix chạy như DaemonSet — 1 instance mỗi Node, luôn watch K8s API:

- Nhận event `NetworkPolicy` / `Pod` / `Node` thay đổi → tính toán lại rules.
- Dịch policy thành **iptables chains** (hoặc eBPF programs) và cập nhật atomic.
- Toàn bộ quá trình: event nhận → iptables update **< 100ms**.
- Không cần restart Pod, Node, hay DaemonSet.

Chain hierarchy Felix tạo:

- `FORWARD` → `cali-FORWARD` → `cali-from-wl-dispatch` → `cali-fw-<id>` (egress)
- `cali-to-wl-dispatch` → `cali-tw-<id>` (ingress)

Mỗi lần apply `NetworkPolicy`, Felix tự đồng bộ — không có hành động thủ công nào cần thiết.

BIRD & Typha

BIRD — BGP daemon, chạy cùng Felix trong Pod `calico-node` :

- Peer với BIRD của tất cả Nodes khác (full mesh) hoặc Route Reflector.
- Quảng bá: *"Pod subnet `10.244.1.0/24` nằm ở Node này"*.
- Cài routes nhận được vào kernel routing table.
- Dừng khi chạy BGP mode (không overlay). Với VXLAN: vẫn chạy nhưng không cần thiết.

Typha — cache layer tùy chọn:

- Đứng giữa K8s API Server và tất cả Felix instances.
- Mỗi Felix chỉ kết nối đến Typha thay vì trực tiếp API Server.
- **Tigera Operator tự bật Typha khi node count > 3** (tùy version).
- Cluster nhỏ (≤ 3 nodes): không cần — Felix kết nối thẳng API Server.

Lab Time: Giải phẫu Calico Architecture

Thực hành theo thứ tự trong `lab-guide.md`:

1. **TN1 — Felix log real-time**: mở 2 terminal song song — terminal 1 watch Felix log, terminal 2 apply NetworkPolicy mới → thấy Felix xử lý update trong ms.
2. **TN2 — iptables chains**: liệt kê `cali-*` chains, xem `cali-FORWARD`, drill vào `cali-tw-<hash>` để thấy ACCEPT/DROP rule tương ứng với policy.
3. **TN3 — calicoctl**: cài binary, dùng `get workloadendpoint`, `get ippool`, `get felixconfig`, `node status`.
4. **TN4 — Typha**: kiểm tra Typha có chạy không, đếm Felix connections, xem Operator quyết định bật Typha khi nào.

👉 Làm theo `lab-guide.md`

Key Takeaways

- **Felix** là trái tim: event-driven, dịch NetworkPolicy → iptables/eBPF < 100ms, không polling.
- **BIRD** quảng bá Pod routes qua BGP — cần thiết khi chạy BGP mode, không quan trọng với VXLAN.
- **Typha** chỉ cần khi cluster lớn (> 50 nodes) để giảm tải K8s API Server.
- **calicoctl** là `kubectl` cho Calico objects — dùng để debug endpoint, IP pool, BGP session.

```
kubectl -n calico-system logs daemonset/calico-node -c calico-node # Felix logs
calicoctl get workloadendpoint          # Pods được Calico quản lý
calicoctl node status                   # BGP peers + health
```

Tập tiếp theo: *iptables vs eBPF dataplane — khi nào upgrade và đánh đổi là gì?*